Advanced Notification Routing

> **Series:** WFLOW | **Notebook:** 4 of 9 | **Created:** January 2026 | **Last Updated:** 01/28/2026

Intelligent Alert Routing

Not all alerts should go to everyone. This notebook covers conditional routing based on severity, team ownership, time of day, and escalation patterns.

Table of Contents

1. Routing Strategies
2. Conditional Expressions
3. Routing by Severity
4. Routing by Team/Service
5. Time-Based Routing
6. Escalation Patterns
7. Multi-Channel Strategy

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Platform subscription
Permissions	`automation:workflows:write`
Prior Knowledge	**WFLOW-01** through **WFLOW-03**
Connections	Slack and/or Teams connections configured

1. Routing Strategies

Why Route Alerts?

Problem	Impact	Solution
Alert fatigue	Teams ignore alerts	Route only relevant alerts
Slow response	Wrong team notified	Route to owners
Off-hours noise	Sleep disruption	Time-based routing
Missed escalation	Unacknowledged alerts	Auto-escalation

Routing Dimensions

Dimension	Route Based On	Example
Severity	Problem severity level	Critical → PagerDuty, Low → Slack
Team	Entity tags, management zones	`team:checkout` → #checkout-alerts
Service	Service name or entity ID	Payment service → payments team
Time	Hour of day, day of week	Weekends → on-call only
Environment	prod/staging/dev tags	Prod → immediate, Dev → daily digest

2. Conditional Expressions

Conditions control which tasks execute. They use Jinja expressions returning boolean values.

Defining Conditions

```
conditions:
  - name: is_critical
    expression: '{{ event()["severity"] == "CRITICAL" }}'

  - name: is_production
    expression: '{{ "prod" in event()["management_zones"] }}'

  - name: is_checkout_team
    expression: '{{ "team:checkout" in event().get("tags", []) }}'
```

Using Conditions on Tasks

```
tasks:
  - name: pagerduty_alert
    type: dynatrace.pagerduty:create-incident
    conditions:
      - is_critical
      - is_production
    # Task runs only if BOTH conditions are true (AND logic)
```

Negating Conditions

```
tasks:
  - name: slack_only_for_non_critical
    type: dynatrace.slack:message
    conditions:
      - not is_critical
    # Task runs when is_critical is FALSE
```

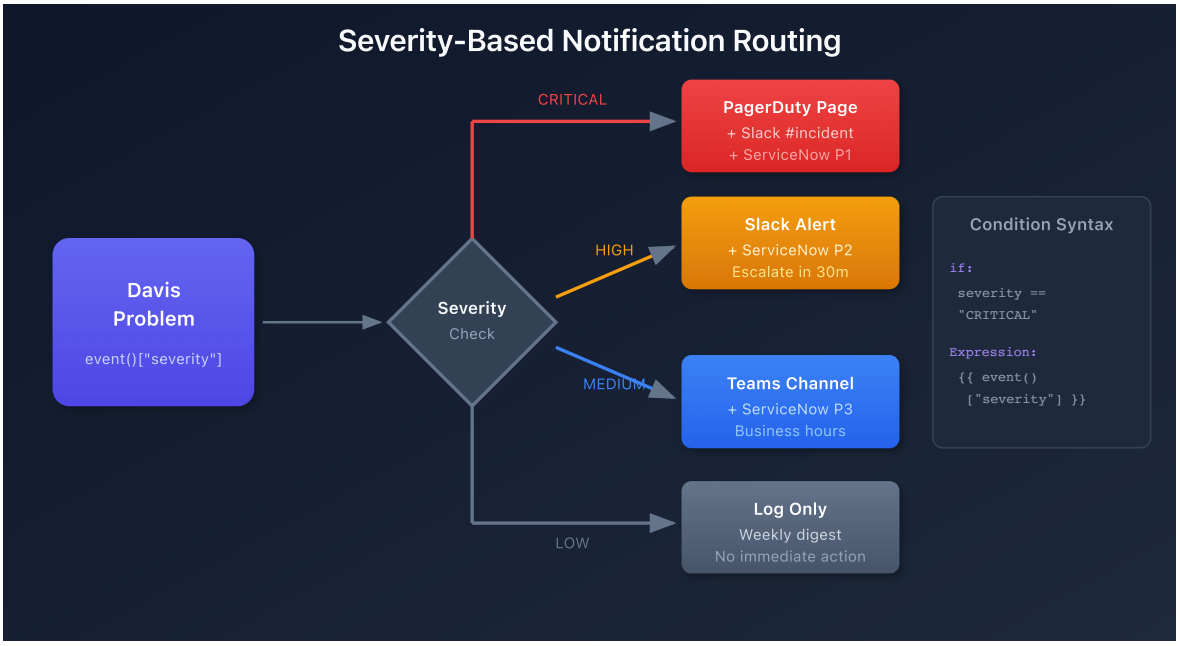
Complex Expressions

```
# AND within expression
{{ event()["severity"] == "CRITICAL" and "prod" in event()
["management_zones"] }}

# OR within expression
{{ event()["severity"] in ["CRITICAL", "HIGH"] }}

# Check if field exists
{{ event().get("root_cause_entity_id") is not none }}
```

3. Routing by Severity

Severity-Based Routing Pattern

Severity	Actions
CRITICAL	PagerDuty + Slack #urgent + Email
HIGH	Slack #alerts + Email
MEDIUM	Slack #alerts
LOW	Slack #alerts (business hours only)



Workflow Configuration

```
conditions:
  - name: is_critical
    expression: '{{ event()["severity"] == "CRITICAL" }}'
  - name: is_high
    expression: '{{ event()["severity"] == "HIGH" }}'
  - name: is_medium_or_low
    expression: '{{ event()["severity"] in ["MEDIUM", "LOW"] }}'

tasks:
  # CRITICAL: Page on-call + Slack urgent + Email
  - name: pagerduty_critical
    type: dynatrace.pagerduty:create-incident
    conditions: [is_critical]
    input:
      connection: pagerduty-prod
      severity: critical
      summary: "{{ event()['title'] }}"

  - name: slack_urgent
    type: dynatrace.slack:message
    conditions: [is_critical]
    input:
      connection: slack-production
      channel: "#alerts-urgent"
      message: ":rotating_light: *CRITICAL* {{ event()['title'] }}"

  # HIGH: Slack alerts + Email
  - name: slack_high
    type: dynatrace.slack:message
    conditions: [is_high]
    input:
      channel: "#alerts-production"
      message: ":warning: *HIGH* {{ event()['title'] }}"

  # MEDIUM/LOW: Slack only
  - name: slack_low
    type: dynatrace.slack:message
    conditions: [is_medium_or_low]
    input:
      channel: "#alerts-production"
      message: ":information_source: *{{ event()['severity'] }}* {{ event()
['title'] }}"
```

4. Routing by Team/Service

Tag-Based Team Routing

Entities tagged with `team:checkout`, `team:payments`, etc.

```
conditions:
  - name: is_checkout_team
    expression: |
      {% set tags = event().get("tags", []) %}
      {{ "team:checkout" in tags or "team:cart" in tags }}

  - name: is_payments_team
    expression: '{{ "team:payments" in event().get("tags", []) }}'

  - name: is_platform_team
    expression: '{{ "team:platform" in event().get("tags", []) }}'

tasks:
  - name: notify_checkout
    type: dynatrace.slack:message
    conditions: [is_checkout_team]
    input:
      channel: "#checkout-alerts"
      message: "{{ event()['title'] }}"

  - name: notify_payments
    type: dynatrace.slack:message
    conditions: [is_payments_team]
    input:
      channel: "#payments-alerts"
      message: "{{ event()['title'] }}"

  - name: notify_platform
    type: dynatrace.slack:message
    conditions: [is_platform_team]
    input:
      channel: "#platform-alerts"
      message: "{{ event()['title'] }}"
```

Management Zone Routing

```
conditions:
  - name: is_us_region
    expression: '{{ "US-East" in event()["management_zones"] or "US-West" in
event()["management_zones"] }}'

  - name: is_eu_region
    expression: '{{ "EU-West" in event()["management_zones"] }}'

tasks:
  - name: notify_us_team
    conditions: [is_us_region]
    input:
      channel: "#us-oncall"

  - name: notify_eu_team
    conditions: [is_eu_region]
    input:
      channel: "#eu-oncall"
```

Dynamic Channel Selection

Use JavaScript to determine the channel dynamically:

```
export default async function({ event }) {
  const tags = event.tags || [];

  // Find team tag
  const teamTag = tags.find(t =&gt; t.startsWith('team:'));
  const team = teamTag ? teamTag.split(':')[1] : 'platform';

  return {
    channel: `#${team}-alerts`,
    team: team
  };
}
```

5. Time-Based Routing

Business Hours vs Off-Hours

```
conditions:
  - name: is_business_hours
    expression: |
      {% set hour = now().hour %}
      {% set weekday = now().weekday() %}
      {{ weekday &lt; 5 and hour &gt;= 9 and hour &lt; 17 }}

  - name: is_off_hours
    expression: |
      {% set hour = now().hour %}
      {% set weekday = now().weekday() %}
      {{ weekday &gt;= 5 or hour &lt; 9 or hour &gt;= 17 }}

tasks:
  # Business hours: Slack channel
  - name: slack_business_hours
    type: dynatrace.slack:message
    conditions: [is_business_hours]
    input:
      channel: "#alerts-production"
      message: "{{ event()['title'] }}"

  # Off-hours: PagerDuty for CRITICAL only
  - name: pagerduty_off_hours
    type: dynatrace.pagerduty:create-incident
    conditions: [is_off_hours, is_critical]
    input:
      connection: pagerduty-prod
      summary: "[Off-Hours] {{ event()['title'] }}"

  # Off-hours non-critical: Queue for morning
  - name: slack_queue
    type: dynatrace.slack:message
    conditions: [is_off_hours, not is_critical]
    input:
      channel: "#alerts-queue"
      message: ":moon: *Queued for review* {{ event()['title'] }}"
```

Weekend-Only Routing

```
conditions:
  - name: is_weekend
    expression: '{{ now().weekday() &gt;= 5 }}'

tasks:
  - name: weekend_oncall
    conditions: [is_weekend, is_critical]
    input:
      # Page weekend on-call rotation
      routing_key: "{{ env.PD_WEEKEND_ROUTING_KEY }}"
```

6. Escalation Patterns

Timed Escalation

Escalate if not acknowledged within a time window.

```
tasks:
  # Step 1: Initial notification
  - name: initial_slack
    type: dynatrace.slack:message
    input:
      channel: "#alerts-production"
      message: ":warning: {{ event()['title'] }} - Acknowledge within 15 min"

  # Step 2: Wait 15 minutes
  - name: wait_for_ack
    type: dynatrace.automations:wait
    dependsOn: [initial_slack]
    input:
      duration: "15m"

  # Step 3: Check if still open
  - name: check_problem_status
    type: dynatrace.automations:run-javascript
    dependsOn: [wait_for_ack]
    input:
      script: |
        import { eventsClient } from '@dynatrace-sdk/client-classic-
environment-v2';

        export default async function({ event }) {
          const problemId = event.display_id;
          // Check if problem is still open
          // Return escalate: true if needs escalation
          return { escalate: event.status === 'OPEN' };
        }

  # Step 4: Escalate to PagerDuty
  - name: escalate_pagerduty
    type: dynatrace.pagerduty:create-incident
    dependsOn: [check_problem_status]
    conditions:
      - '{{ result("check_problem_status").escalate }}'
    input:
      severity: high
      summary: "[ESCALATED] {{ event()['title'] }} - No acknowledgment in 15
min"
```

Multi-Tier Escalation

```
0 min  → Slack channel notification
15 min → Email to team lead
30 min → PagerDuty to on-call
60 min → PagerDuty to manager
```

7. Multi-Channel Strategy

Recommended Channel Matrix

Severity	Environment	Channel(s)
CRITICAL	Production	PagerDuty + Slack urgent + Email
CRITICAL	Staging	Slack alerts
HIGH	Production	Slack alerts + Email
HIGH	Staging	Slack alerts
MEDIUM	Production	Slack alerts
MEDIUM	Staging	Slack (daily digest)
LOW	Any	Slack (weekly digest)

Complete Multi-Channel Workflow

```
conditions:
  - name: is_critical
    expression: '{{ event()["severity"] == "CRITICAL" }}'
  - name: is_high_or_above
    expression: '{{ event()["severity"] in ["CRITICAL", "HIGH"] }}'
  - name: is_production
    expression: '{{ "Production" in event().get("management_zones", []) }}'
  - name: is_business_hours
    expression: '{{ now().weekday() &lt; 5 and now().hour &gt;= 9 and
now().hour &lt; 17 }}'

tasks:
  # PagerDuty: Critical + Production
  - name: pagerduty
    type: dynatrace.pagerduty:create-incident
    conditions: [is_critical, is_production]

  # Slack Urgent: Critical + Production
  - name: slack_urgent
    type: dynatrace.slack:message
    conditions: [is_critical, is_production]
    input:
      channel: "#alerts-urgent"

  # Slack Standard: High or above + Production
  - name: slack_standard
    type: dynatrace.slack:message
    conditions: [is_high_or_above, is_production]
    input:
      channel: "#alerts-production"

  # Email: Critical or (High + Production + Business Hours)
  - name: email_alert
    type: dynatrace.email:send
    conditions:
      - '{{ event()["severity"] == "CRITICAL" or (event()["severity"] ==
"HIGH" and "Production" in event().get("management_zones", [])) }}'
```

Query Workflow Routing Effectiveness

```
// Workflow executions by trigger severity
fetch events, from: now() - 7d
| filter event.type == "automation.workflow.execution"
| summarize executions = count(), by:{workflow.name, trigger.severity}
| sort executions desc
| limit 30
```

```
// Task execution distribution by notification channel
fetch events, from: now() - 7d
| filter event.type == "automation.task.execution"
| filter contains(task.type, "slack") or contains(task.type, "msteams") or
contains(task.type, "pagerduty") or contains(task.type, "email")
| summarize
    total = count(),
    succeeded = countIf(task.status == "SUCCEEDED"),
    by:{task.type}
| fieldsAdd success_rate = round(100.0 * succeeded / total, decimals: 2)
| sort total desc
```

```
// Skipped tasks (conditions not met)
fetch events, from: now() - 24h
| filter event.type == "automation.task.execution"
| filter task.status == "SKIPPED"
| summarize skipped_count = count(), by:{workflow.name, task.name}
| sort skipped_count desc
| limit 20
```

Next Steps

With routing configured, integrate with incident management:

Recommended Path

1. **WFLOW-05: PagerDuty & ServiceNow** - Create incidents automatically
2. **WFLOW-06: Custom Templates** - Rich message formatting
3. **WFLOW-07: Problem-Triggered Remediation** - Auto-remediation

Key Takeaways

- **Conditions** control task execution using Jinja expressions
- **Severity routing** ensures appropriate response levels
- **Team routing** uses tags or management zones
- **Time-based routing** reduces off-hours noise
- **Escalation patterns** prevent missed alerts
- Test conditions with On-Demand trigger before production

Summary

In this notebook, you learned:

- Why and when to route alerts conditionally
- Conditional expression syntax and operators
- Severity-based routing patterns
- Team and management zone routing
- Time-based (business hours) routing
- Escalation with wait and check patterns
- Multi-channel notification strategy

References

- Workflow Conditions
- Jinja Expressions
- Wait Action
- Task Dependencies

Dynatrace documentation.